

FANTASTIC HEAPS

... and where to find them.
Datastructure Love!

Florian Sihler

September 21, 2026
SP, Ulm University

FANTASTIC HEAPS

... and where to find them.
Datastructure Love!

Florian Sihler



Officially supported by the Pingu-Foundation for Emotional Support. There will be light and there will be cute waddlers!

September 21, 2026
SP, Ulm University

2

Disclaimer

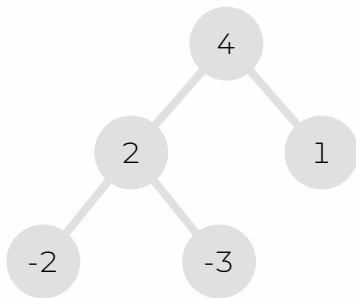
- This “heap” has nothing to do with the one from the recursion episode!
- We restrict ourselves to heap-*trees*
- We ignore rotations
- These slides were made in ~ 3.5 hours hence condensed

[6]: *Episode-Recursion* Sihler, 2021

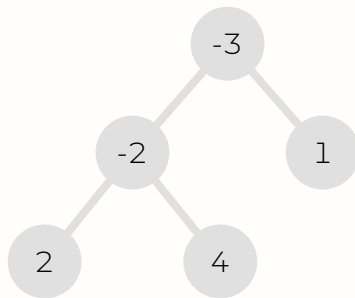
[5]: *BTk-Package*, tikzpingus
Sihler, 2021

3

The Poster Children



- **Max-Heap**
parents $\geq$ children

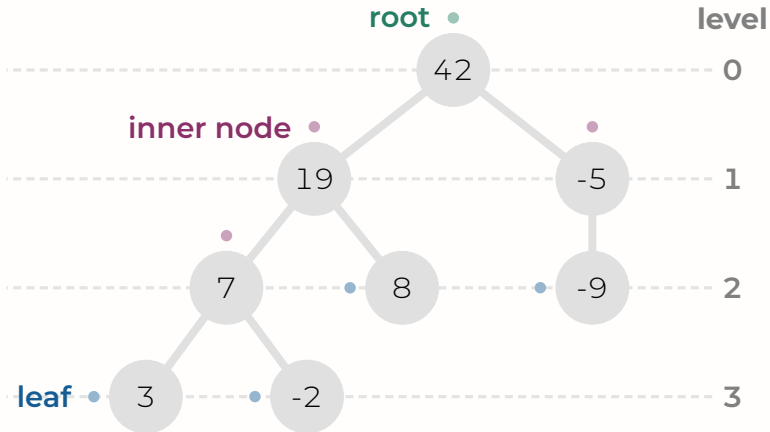


- Min-Heap**
parents $\leq$ children

4

The Anatomy

So leaves are nodes without any further child nodes.



The lines between the nodes are simply called "edges".





- For us a tree is
 - a graph $G = (V, E)$ with $|V|$ nodes and $|E|$ edges
 - V and E are finite
 - acyclic & connected (“maximally acyclic”)
- Even a binary tree!
 - Every node has at most two successors
 - We focus on balanced ones
- Trees can be seen as a recursive structure

[6]: *Episode-Recursion* Sihler, 2021

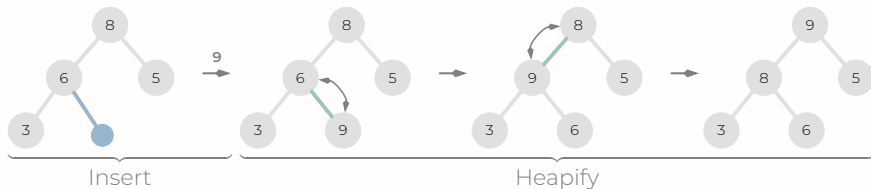
[2]: *The average height of binary
trees and other simple trees*
Flajolet, 1982

Insert/Put

If the lowest level is full, we simply create a new one!



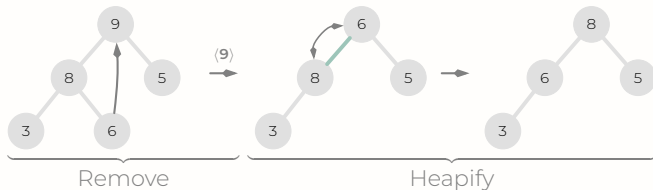
- On the lowest level, as far to the left as possible
- Afterwards: *heapify*



8

Remove/Get

- Replace the topmost element with the “last” one
- Afterwards: *heapify*



9

Further Operations

- Find/Extract Min/Max
- Replace (one node with another)
- Meld/Join

[3]: *Refined complexity analysis for heap operations* Fredman, 1987

[7]: *Heap: A Data Structure for Efficient Programs* Singh, 2013



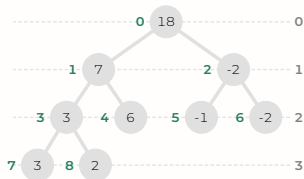
10

Astonishing Insights

Assumptions:

The heap is completely filled up to the last level.
The last level fills up from the left.

- Every filled level i holds 2^i elements
- Numbering by breadth-first traversal allows addressing!
- The left child of n is $2 \cdot n + 1$, the right child $2 \cdot n + 2$
- The parent of n is $\lfloor \frac{n-1}{2} \rfloor$



Hmmmm... That's just array indexing!



11

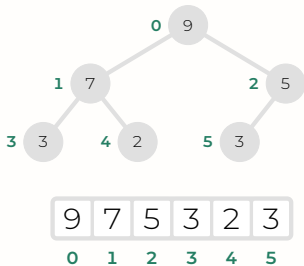
A Heap as an Array

- Heaps usually are “just” arrays
- Helper routines make working with them pleasant:

```
int left(int n) { return 2 * n + 1; }  
int right(int n) { return 2 * n + 2; }  
int parent(int n) { return (n - 1) / 2; }
```

Tip: with arrays, *swap*
is usually extremely
helpful as well!

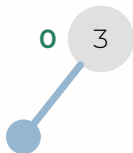
- Checks certainly do not hurt!



12.1

A Descending Example

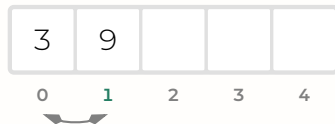
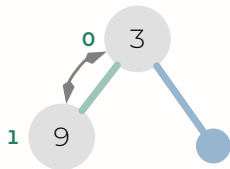
[3, 9, 8, 2, 12]



12.2

A Descending Example

[3, 9, 8, 2, 12]



Preliminaries



Kinds of Heaps



Operations



Arrays



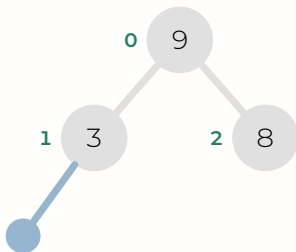
Wrap-Up



12.3

A Descending Example

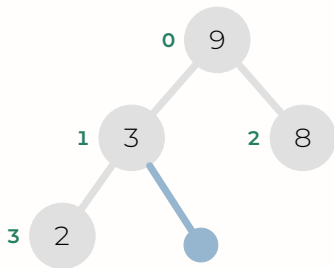
[3, 9, 8, 2, 12]



12.4

A Descending Example

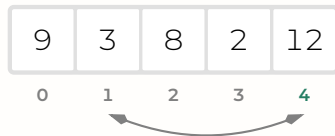
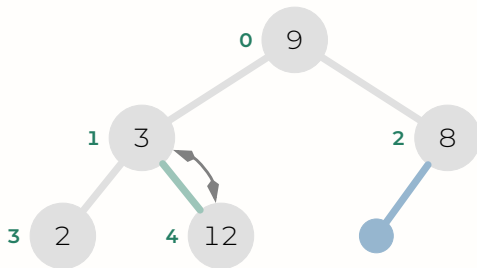
[3, 9, 8, 2, 12]



12.5

A Descending Example

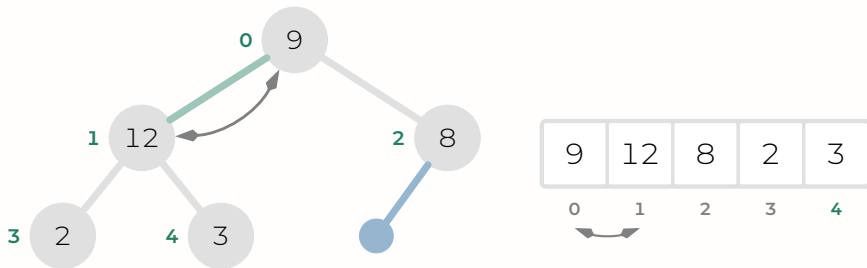
[3, 9, 8, 2, 12]



12.6

A Descending Example

[3, 9, 8, 2, 12]



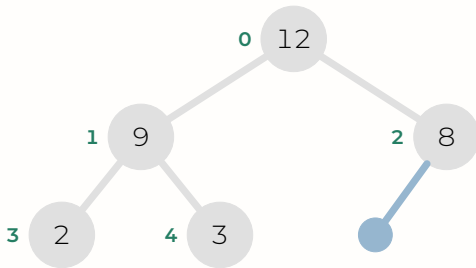
12.7

A Descending Example

If the heap is built with *insert* only, at most the new element violates the heap property. On top of that it stays nicely balanced.



[3, 9, 8, 2, 12]



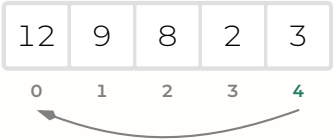
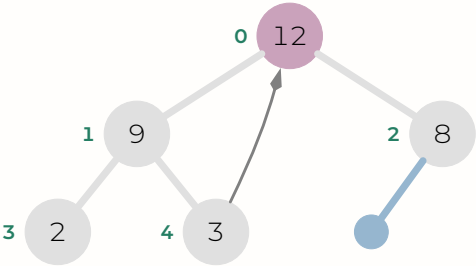


Removal, for a change,
now with “down-heapify”.

12.8

A Descending Example

[3, 9, 8, 2, 12]



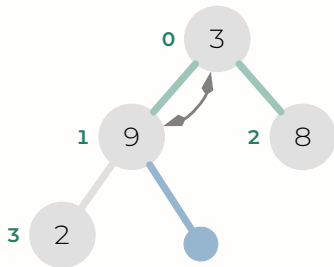
12.9

A Descending Example

Removal, for a change,
now with “down-heapify”.
With several conflicts: swap with
the *more extreme* element.



[3, 9, 8, 2, 12]



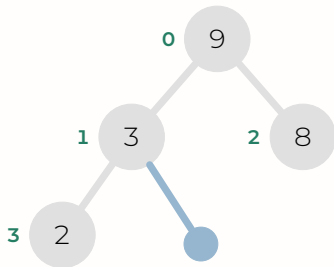
12.10

A Descending Example

Removal, for a change,
now with “down-heapify”.
With several conflicts: swap with
the *more extreme* element.



[3, 9, 8, 2, 12]



13

For Practice

- Visualize *insert* $[3, 5, 8, 3, -5, 6]$ (in this order) in a *min*-heap. Best done simultaneously in the tree and in the array.
- Now visualize the removal until the heap is empty
- For these and other data structures, working through them *in practice* helps!

[4]: *Binary Search Trees*
GeeksforGeeks, 2020

[1]: *Exercises for Algorithms and
Data Structures* Carzaniga, 2016



- [1] Antonio Carzaniga. “Exercises for Algorithms and Data Structures”. 2016
- [2] Philippe Flajolet and Andrew Odlyzko. “The average height of binary trees and other simple trees”. 1982
- [3] Michael L. Fredman and Thomas H. Spencer. “Refined complexity analysis for heap operations”. 1987
- [4] GeeksforGeeks. *Binary Search Trees*. 2020
- [5] Florian Sihler. *L^AT_EX*-Package, *tikzpingus*. 2021
- [6] Florian Sihler. *Episode-Recursion*. 2021
- [7] Sanjay Singh. “Heap : A Data Structure for Efficient Programs”. 2013



Florian Sihler

Ulm, September 21, 2026